

Cây tìm kiếm và cân bằng (Search Trees and Balance)

PHỤ THUỘC B/08 cho phía đối lập — băm không thứ tự; A/04 cho bất biến, vì cân bằng chính là một bất biến được duy trì sau mỗi thao tác.

TÓM TẮT

Cây tìm kiếm là câu trả lời cho câu hỏi mà băm không trả lời được: *giữ dữ liệu có thứ tự với chi phí cập nhật rẻ*. Ý tưởng gốc rất đơn giản — mỗi nút chia dữ liệu thành nhỏ hơn và lớn hơn — nhưng nó thoái hoá thành danh sách nếu dữ liệu vào theo thứ tự sắp sẵn, và toàn bộ chương này là các cách khác nhau để chống lại sự thoái hoá đó: quay để giữ chiều cao (AVL, đỏ-đen), làm nút to ra cho hợp với đĩa (B-cây), hoặc dùng ngẫu nhiên để cân bằng gần như miễn phí (treap, danh sách bỏ qua). Chương cũng nói về *tăng cường* — gắn thêm thông tin vào mỗi nút để cây trả lời được câu hỏi mới, kỹ thuật đáng giá nhất ở đây. Nếu chỉ nhớ một điều: chiều cao cây là bất biến cần duy trì, và mọi cấu trúc chỉ khác nhau ở cách trả tiền cho bất biến đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **cây tìm kiếm nhị phân (BST)** — cây nhị phân trong đó mọi nút ở cây con trái nhỏ hơn nút gốc, mọi nút ở cây con phải lớn hơn. Duyệt giữa cho dãy tăng dần.
- **chiều cao cây** — số cạnh trên đường dài nhất từ gốc tới lá; quyết định chi phí mọi thao tác. Cây cân bằng có chiều cao $O(\log n)$, cây thoái hoá có chiều cao $O(n)$.
- **phép quay (rotation)** — thao tác cục bộ đổi chỗ một nút với con của nó, giữ nguyên thứ tự nhưng đổi chiều cao. Đây là công cụ cơ bản của mọi cây tự cân bằng.
- **tăng cường (augmentation)** — lưu thêm thông tin ở mỗi nút — kích thước cây con, tổng, cực đại — để trả lời được truy vấn mới mà vẫn giữ $O(\log n)$.
- **B-cây** — cây tìm kiếm nhiều nhánh, mỗi nút chứa nhiều khoá, thiết kế để mỗi nút vừa một khối đĩa. Cơ sở của mọi chỉ mục cơ sở dữ liệu.
- **treap** — cây kết hợp tính chất BST theo khoá và tính chất heap theo một độ ưu tiên *ngẫu nhiên*; cân bằng theo nghĩa kỳ vọng mà không cần luật quay phức tạp.

CÂU HỎI MỞ ĐẦU

- Q1. Cây tìm kiếm nhị phân đơn giản chạy $O(\log n)$ “trung bình”. Vì sao trong thực tế nó lại nguy hiểm, và tình huống nào làm nó tệ nhất?
- Q2. Cây đỏ-đen và AVL cùng cho $O(\log n)$. Vì sao thư viện chuẩn thường chọn đỏ-đen còn cơ sở dữ liệu lại chọn B-cây?
- Q3. Treap dùng số ngẫu nhiên để cân bằng và không có luật quay phức tạp nào. Vì sao nó vẫn đảm bảo được chiều cao $O(\log n)$?
- Q4. Vì sao “tăng cường” là kỹ thuật đáng giá nhất của chương này, hơn cả việc thuộc luật cân bằng?
- Q5. Khi nào một cây cân bằng $O(\log n)$ lại thua một mảng đã sắp $O(n)$ trên máy thật?

Chương trước cho ta tra cứu $O(1)$ nhưng không có thứ tự. Rất nhiều bài toán cần thứ tự: tìm phần tử gần nhất, liệt kê mọi phần tử trong một khoảng, tìm phần tử nhỏ nhất lớn hơn x , hỏi phần tử đứng thứ k . Nếu dữ liệu tĩnh thì mảng đã sắp cộng tìm kiếm nhị phân giải hết — rẻ và

nhANH. Bài toán chỉ trở nên khó khi dữ liệu *vừa cần thứ tự vừa thay đổi liên tục*, vì chèn vào mảng đã sắp tốn $O(n)$.

Cây tìm kiếm là câu trả lời cho đúng tình huống đó, và ý tưởng thì hiển nhiên: thay vì giữ dữ liệu thành một dãy, hãy giữ nó thành một cấu trúc phân nhánh, để việc chèn chỉ động tới một đường từ gốc xuống lá. Cái không hiển nhiên — và là toàn bộ nội dung của chương — là làm sao giữ cho đường đó ngắn.

1.1 Bài toán thoái hoá và ý nghĩa của cân bằng

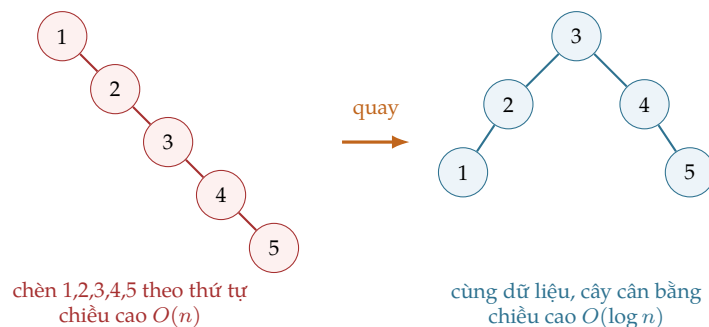
Cây tìm kiếm nhị phân thô sơ chèn phần tử mới bằng cách đi từ gốc, rẽ trái nếu nhỏ hơn, rẽ phải nếu lớn hơn, cho tới khi gặp chỗ trống. Nếu dữ liệu tới theo thứ tự ngẫu nhiên, chiều cao kỳ vọng là $O(\log n)$ và mọi thứ đều tốt.

Nhưng nếu dữ liệu tới theo thứ tự *đã sắp* — tình huống cực kỳ phổ biến trong thực tế, vì dữ liệu thật thường có trật tự sẵn — thì mọi phần tử đều rẽ phải và cây thành một danh sách liên kết dài n . Mọi thao tác thành $O(n)$, và cấu trúc mà bạn chọn để tăng tốc lại làm chậm chương trình so với mảng thường.

Đây là cùng một câu chuyện với chương B/08: một cấu trúc nhanh “trung bình” nhưng có trường hợp xấu mà dữ liệu thực tế lại rất dễ rơi vào. Và có hai hướng chữa, hết như ở đó.

Hướng tất định: thêm một bất biến về hình dạng và duy trì nó sau mỗi thao tác. AVL yêu cầu chiều cao hai cây con của mọi nút chênh nhau không quá 1 — rất chặt, nên cây thấp và tìm kiếm nhanh, nhưng phải quay nhiều khi chèn/xoá. Cây đỏ-đen yêu cầu lỏng hơn — không đường nào từ gốc tới lá dài quá hai lần đường ngắn nhất — nên cây cao hơn một chút nhưng số phép quay ít hơn. Sự khác nhau đó giải thích lựa chọn của các thư viện: khi tỉ lệ chèn/xoá cao, đỏ-đen tổng thể rẻ hơn.

Hướng ngẫu nhiên: không ép hình dạng, mà làm cho hình dạng *không phụ thuộc thứ tự dữ liệu vào*. Đây là ý tưởng của treap và danh sách bỏ qua, và mục 3 sẽ nói kỹ vì nó rất đẹp.



Hình 1.1. Cùng năm khoá, hai hình dạng. Cây trái là kết quả của việc chèn dữ liệu đã sắp sẵn — trường hợp rất hay gặp trong thực tế, không phải trường hợp hiếm. Mọi cấu trúc trong chương này chỉ làm một việc: ngăn hình dạng bên trái xuất hiện, bằng phép quay hoặc bằng ngẫu nhiên hoá.

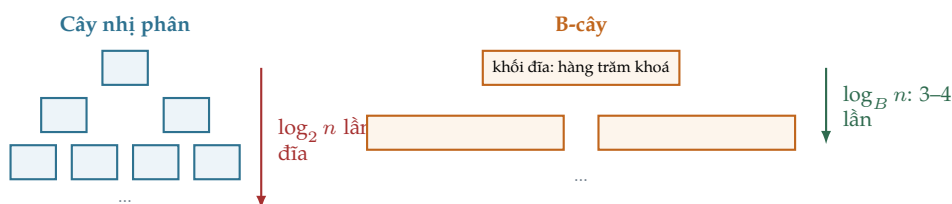
1.2 Phép quay: thao tác cơ bản duy nhất

Mọi cây tự cân bằng đều dựa trên một thao tác duy nhất là phép quay: đổi chỗ một nút với con của nó, nối lại ba cây con cho đúng. Điều quan trọng nhất về phép quay — và là chỗ đáng dừng lại một phút — là nó *bảo toàn thứ tự duyệt giữa* nhưng *thay đổi chiều cao*. Nói theo ngôn ngữ chương A/04: phép quay giữ nguyên bất biến ngữ nghĩa (tính chất BST) trong khi sửa bất biến cấu trúc

(chiều cao). Đó chính là lý do nó là công cụ đúng: bạn cần sửa hình dạng mà không được làm hỏng nội dung.

Từ đó, các cấu trúc chỉ khác nhau ở *khi nào quay* và *quay bao nhiêu*: AVL quay khi chênh lệch chiều cao vượt 1; đỏ-đen quay và đổi màu theo một bộ luật dài hơn nhưng ít quay hơn; treap quay để khôi phục tính chất heap theo độ ưu tiên ngẫu nhiên; splay quay đưa nút vừa truy cập lên gốc.

Cây splay đáng nhắc riêng vì nó đại diện cho một triết lý khác[1]: không đảm bảo gì cho từng thao tác, chỉ đảm bảo $O(\log n)$ *khẩu hao*, đổi lại có tính chất rất hay là các phần tử hay truy cập tự động nổi lên gần gốc. Trong hệ thống có tính cục bộ truy cập cao, điều đó làm nó nhanh hơn cây cân bằng chặt. Nhược điểm: mỗi lần *đọc* cũng làm thay đổi cấu trúc, nên nó khó dùng trong môi trường nhiều luồng.



Hình 1.2. Đổi đơn vị chi phí thì đổi hình dạng cấu trúc. Khi chi phí thật là *lần đọc khối* chứ không phải phép so sánh, việc đăng nào cũng phải trả tiền cho một lần đọc khiến ta muốn đọc thật nhiều khoá một lúc — và cây từ nhị phân biến thành nhiều nhánh với nút vừa một khối đĩa.

1.3 Ngẫu nhiên hoá: cách rẻ nhất để cân bằng

Treap là một trong những ý tưởng đẹp nhất trong cấu trúc dữ liệu, và nó chỉ cần một câu để phát biểu: mỗi nút có khoá (do dữ liệu quyết định) và một *độ ưu tiên ngẫu nhiên* (do bạn sinh ra); cây được giữ sao cho theo khoá thì nó là BST, còn theo độ ưu tiên thì nó là heap.

Vì sao điều đó cho cân bằng? Vì hình dạng của một treap chỉ phụ thuộc vào *thứ tự các độ ưu tiên*, không phụ thuộc thứ tự chèn. Mà độ ưu tiên do bạn sinh ngẫu nhiên, nên hình dạng của treap giống hệt hình dạng của một BST được chèn theo thứ tự ngẫu nhiên — và chiều cao kỳ vọng của cây đó là $O(\log n)$. Nói cách khác: bạn không thể kiểm soát thứ tự dữ liệu tới, nhưng bạn có thể tự tạo ra một thứ tự ngẫu nhiên khác và để hình dạng phụ thuộc vào thứ tự đó.

Đây chính xác là ý tưởng của băm phổ dụng ở chương B/08 và của chốt ngẫu nhiên trong quick-sort ở chương C/14, xuất hiện lần thứ ba trong quyển sách này. Nếu bạn thấy mẫu ấy lặp lại thì đó không phải trùng hợp mà là một nguyên lý: *khi trường hợp xấu phụ thuộc vào dữ liệu, hãy chuyển sự phụ thuộc đó sang một nguồn ngẫu nhiên mà bạn kiểm soát*. Chương C/20 dành trọn để khai thác nguyên lý này.

Danh sách bỏ qua (skip list) là một hiện thực hoá khác của cùng ý tưởng, dùng nhiều tầng danh sách liên kết với xác suất lên tầng là $1/2$. Nó không phải cây, nhưng cho cùng đảm bảo kỳ vọng, dễ cài hơn cây cân bằng và dễ làm phiên bản nhiều luồng — đó là lý do một số hệ cơ sở dữ liệu chọn nó cho phần dữ liệu nằm trong bộ nhớ.

1.4 Tăng cường: kỹ thuật đáng giá nhất của chương

Đây là mục mà nếu chỉ đọc một mục thì nên đọc mục này. Ý tưởng: gắn thêm thông tin vào mỗi nút, miễn là thông tin đó *tính được từ hai con*, thì bạn giữ được nó qua mọi thao tác với chi phí không đổi, và cây trả lời được câu hỏi hoàn toàn mới.

Ví dụ quan trọng nhất: lưu ở mỗi nút *kích thước cây con* của nó. Với thông tin đó, cây trả lời được hai câu hỏi mà băm và heap đều chịu: “phần tử đứng thứ k là ai” và “có bao nhiêu phần tử nhỏ hơn x ” — cả hai trong $O(\log n)$, bằng cách đi xuống từ gốc và trừ dần. Cấu trúc này gọi là cây thống kê thứ tự, và nó là công cụ chuẩn cho các bài đếm nghịch thế hoặc đếm phần tử nhỏ hơn trong đoạn.

Ví dụ thứ hai: lưu *giá trị lớn nhất của điểm kết thúc* trong cây con, ta được cây khoảng — trả lời “có đoạn nào chứa điểm x không” hoặc “liệt kê mọi đoạn giao với $[l, r]$ ”. Đây là cấu trúc dùng trong hình học tính toán (D/22) và trong các hệ thống lập lịch.

Quy tắc chung đáng nhớ: *thông tin tăng cường phải tổng hợp được từ các con trong $O(1)$* . Kích thước, tổng, cực đại, cực tiểu đều thỏa; “phần tử phổ biến nhất trong cây con” thì không, vì không suy ra được từ hai con. Cùng điều kiện này sẽ xuất hiện lại ở chương B/10 với cây phân đoạn, và ở đó ta sẽ thấy nó chính là điều kiện “phép toán có tính kết hợp”.

1.5 B-cây: khi chi phí thật không phải là số phép so sánh

Mọi phân tích trên đếm số phép so sánh. Nhưng nếu dữ liệu nằm trên đĩa, chi phí thật là số lần đọc khối — và một lần đọc đĩa đắt hơn hàng triệu phép so sánh trong bộ nhớ. Với mô hình chi phí đó, cây nhị phân là lựa chọn tệ: chiều cao $\log_2 n$ nghĩa là $\log_2 n$ lần đọc đĩa, mà với một tỉ bản ghi thì đó là 30 lần đọc cho một truy vấn.

B-cây giải quyết bằng cách làm nút to ra: mỗi nút chứa hàng trăm khoá và vừa đúng một khối đĩa. Chiều cao trở thành $\log_{100} n$, tức là ba tới bốn tầng cho một tỉ bản ghi. Trong nút thì tìm kiếm bằng so sánh — rẻ, vì nút đã nằm trong bộ nhớ sau khi đọc.

Đây là ví dụ rõ nhất trong cả quyển sách cho luận điểm ở chương A/03: *đổi mô hình chi phí thì đổi luôn cấu trúc tối ưu*. Cùng bài toán, cùng yêu cầu, nhưng khi đơn vị chi phí chuyển từ “phép so sánh” sang “lần chuyển khối” thì lời giải tốt nhất đổi hẳn hình dạng. Và điều đó vẫn đúng khi dữ liệu nằm trong RAM: vì bộ nhớ cũng có phân cấp cache, các cấu trúc “B-cây thu nhỏ” với nút vừa một dòng cache thường nhanh hơn cây nhị phân thuần túy.

Cấu trúc	Đảm bảo	Cài đặt	Chọn khi
AVL	$O(\log n)$ chặt, cây thấp	Khó	Đọc nhiều hơn ghi rất nhiều
Đỏ-đen	$O(\log n)$, ít quay hơn	Khó	Thư viện chuẩn, ghi và đọc cân bằng
Treap	$O(\log n)$ kỳ vọng	Dễ	Tự cài trong thi đấu; cần tách/ghép cây
Danh sách bỏ qua	$O(\log n)$ kỳ vọng	Dễ	Cần phiên bản nhiều luồng
Splay	$O(\log n)$ khẩu hao	Trung bình	Truy cập có tính cục bộ cao
B-cây / B ⁺ -cây	$O(\log_B n)$ lần đọc khối	Khó	Dữ liệu trên đĩa; chỉ mục cơ sở dữ liệu

CẠM BẮY

Trong lập trình thi đấu, đừng cài AVL hay đỏ-đen. Nếu cần cây cân bằng tự viết — thường là để có thao tác tách và ghép, thứ mà thư viện chuẩn không cho — hãy dùng treap: ngắn hơn nhiều lần, ít lỗi hơn hẳn, và đảm bảo kỳ vọng là đủ. Còn nếu chỉ cần tập hợp có thứ tự thông thường thì dùng thẳng thư viện chuẩn; viết lại chỉ tăng nguy cơ sai.

1.6 Splay và câu hỏi mở nổi tiếng nhất của ngành cấu trúc dữ liệu

Bảng ở mục trước xếp splay tree vào ô “ $O(\log n)$ khấu hao, hợp khi truy cập có tính cục bộ”. Dòng đó đúng nhưng giấu mất phần thú vị nhất, và phần ấy đáng kể lại vì nó dạy một cách nghĩ chứ không chỉ một cấu trúc.

Splay tree không lưu *bất kỳ* thông tin cân bằng nào — không chiều cao, không màu, không độ ưu tiên. Luật duy nhất: mỗi lần chạm vào một nút, hãy quay nó lên tận gốc theo một khuôn cố định gồm các bước hai tầng. Sleator và Tarjan[1] chứng minh rằng chỉ luật đó đã đủ cho $O(\log n)$ khấu hao — nhưng kết quả đáng nhớ hơn là hai tính chất mạnh hơn cận ấy.

Tối ưu tĩnh: trên bất kỳ dãy truy cập nào, tổng chi phí của splay chỉ hơn một hằng số lần so với *cây tĩnh tốt nhất* dựng bởi một người biết trước tần suất truy cập của mọi khoá. Nói cách khác, splay đạt được cận entropy của phân phối mà *không hề biết* phân phối đó — nó học từ chính dãy thao tác. *Tính chất tập làm việc*: chi phí chạm vào một khoá phụ thuộc số khoá phân biệt đã được chạm kể từ lần trước chạm nó, chứ không phụ thuộc n . Vì vậy một chương trình liên tục quay lại vài trăm khoá quen sẽ trả giá theo vài trăm chứ không theo hàng triệu — đúng hành vi mà bộ nhớ đệm mong muốn.

Câu hỏi mở nằm ở bước tiếp theo. Sleator và Tarjan phỏng đoán rằng splay không chỉ tối ưu so với cây tĩnh tốt nhất, mà tối ưu trong vòng một hằng số so với *mọi* thuật toán trên cây tìm kiếm nhị phân — kể cả thuật toán được phép quay tùy ý và được biết trước toàn bộ dãy truy cập. Đó là *phỏng đoán tối ưu động*, và sau bốn thập kỷ nó vẫn mở. Tiến bộ lớn đầu tiên là Tango tree của Demaine, Harmon, Iacono và Patrăşcu[2]: một cây trực tuyến có tỉ lệ cạnh tranh $O(\log \log n)$, dựng trên một cận dưới cho *mọi* cây nhị phân. Trước đó, tỉ lệ tốt nhất người ta biết chỉ là $O(\log n)$ — tức là “không tệ hơn việc chẳng thích nghi gì cả”.

Có hai điều đáng mang theo, và cả hai đều lớn hơn bản thân splay tree. Thứ nhất là cách đặt câu hỏi: thay vì hỏi “cấu trúc này tốn bao nhiêu”, ta hỏi “nó tốn bao nhiêu so với lời giải tốt nhất có thể trên cùng dãy thao tác”. Đó chính là phân tích cạnh tranh, thước đo chuẩn của các bài toán trực tuyến ở chương D/24, và ở đây nó được áp cho một cấu trúc dữ liệu. Thứ hai là lời nhắc rằng chữ “tối ưu” luôn kèm một lớp đối thủ và một mô hình: tối ưu so với cây tĩnh, so với mọi cây nhị phân, hay so với mọi cấu trúc bất kỳ — ba câu hỏi khác nhau với ba câu trả lời khác nhau.

Còn phần thực dụng thì ngắn. Đừng cài splay trong kỳ thi: hằng số lớn, một thao tác đơn lẻ có thể tốn $O(n)$, và mọi lần *đọc* đều *ghi* vào cây — điều làm nó khó dùng trong môi trường nhiều luồng và bất lợi với bộ nhớ đệm chỉ đọc. Giá trị của nó với người học nằm ở chỗ khác: nó là bằng chứng rằng một cấu trúc không giữ chút siêu dữ liệu cân bằng nào vẫn có thể tự điều chỉnh theo dữ liệu, và nó là cửa vào cho cả một nhánh nghiên cứu vẫn còn sống.

1.7 Gốc rễ: từ bài toán chiều cao tới bài toán đĩa quay

Cây tìm kiếm nhị phân là ý tưởng cổ và không có tác giả rõ ràng — nó xuất hiện tự nhiên ở nhiều nơi trong thập niên 1950. Điều *có* tác giả rõ ràng là bài toán cân bằng, và cây tự cân bằng đầu tiên ra đời năm 1962 do hai nhà toán học Liên Xô, Adelson-Velsky và Landis, đề xuất — tên AVL là chữ đầu của họ. Đóng góp thực sự của họ không phải cấu trúc mà là *ý niệm*: rằng có thể duy trì một bất biến hình dạng sau mỗi thao tác với chi phí không đổi, và nhờ đó biến một đảm bảo “trung bình” thành đảm bảo tuyệt đối.

B-cây ra đời năm 1970 tại phòng nghiên cứu của Boeing, từ một bài toán hoàn toàn thực tế: chỉ mục cho các tệp lớn nằm trên đĩa quay. Rudolf Bayer và Edward McCreight nhận ra điều mà mô hình đếm phép so sánh che mất — rằng chi phí đọc một khối đĩa gần như không phụ thuộc kích thước khối, nên nếu đằng nào cũng phải trả tiền cho một lần đọc thì hãy đọc thật nhiều khoá một

lúc. Từ nhận xét đó ra cả một họ cấu trúc, và ngày nay gần như mọi hệ cơ sở dữ liệu quan hệ và mọi hệ tệp hiện đại đều dùng một biến thể của nó. Đây là ví dụ đẹp cho việc *đặt lại câu hỏi về đơn vị chi phí* có thể sinh ra cấu trúc mới.

Cây đỏ–đen có nguồn gốc thú vị: nó khởi đầu như một cách biểu diễn B-cây bậc 4 bằng cây nhị phân, rồi được Guibas và Sedgewick trình bày lại vào năm 1978 dưới dạng luật màu mà ta biết ngày nay. Nghĩa là hai cấu trúc trông rất khác nhau thực ra là hai cách nhìn cùng một thứ — một liên hệ đáng nhớ vì nó giải thích vì sao luật đỏ–đen trông tuy tiện: chúng là dấu vết của cấu trúc nhiều nhánh gốc.

Cuối cùng, hai cấu trúc ngẫu nhiên hoá ra đời muộn hơn nhiều và vì một lý do rất người: cây cân bằng tất định quá khó cài đúng. Pugh đề xuất danh sách bỏ qua năm 1990 với luận điểm thẳng thắn rằng nó đơn giản hơn hẳn cây cân bằng mà vẫn cho cùng đảm bảo kỳ vọng; Seidel và Aragon đưa ra treap năm 1996 với cùng tinh thần. Sự chuyển dịch này — chấp nhận đảm bảo kỳ vọng để đổi lấy code ngắn và đúng — là một chủ đề lặp lại trong ngành, và nó gắn với chương C/20.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1950 — BST	Cần cấu trúc vừa tra cứu nhanh vừa chèn được, thay cho mảng đã sắp.	Ý tưởng phân nhánh theo khoá; nhưng chưa ai lo chuyện thoái hoá.
1962 — Adelson-Velsky & Landis	BST thoái hoá thành danh sách khi dữ liệu vào có trật tự.	Cây AVL — cây tự cân bằng đầu tiên; xác lập ý niệm “duy trì bất biến hình dạng sau mỗi thao tác”.
1970 — Bayer & McCreight (Boeing)	Chỉ mục tệp lớn trên đĩa quay; mỗi lần đọc đĩa đắt hơn hàng triệu phép so sánh.	B-cây: đổi đơn vị chi phí từ phép so sánh sang lần đọc khối $\Rightarrow$ nút to, cây thấp. Nền của mọi chỉ mục cơ sở dữ liệu.
1972–78 — Bayer, Guibas & Sedgewick	Muốn ưu điểm của B-cây bậc 4 nhưng ở dạng cây nhị phân.	Cây đỏ–đen; luật màu trông tuy tiện chính là dấu vết của cấu trúc nhiều nhánh gốc.
1985 — Sleator & Tarjan	Có cần đảm bảo cho <i>từng</i> thao tác không, hay tổng thể là đủ?	Splay tree: $O(\log n)$ khẩu hao, tự thích nghi với mẫu truy cập; kèm theo là cả lý thuyết phân tích khẩu hao.
1990–96 — Pugh; Seidel & Aragon	Cây cân bằng tất định quá khó cài đúng.	Danh sách bỏ qua và treap: dùng ngẫu nhiên để đạt cùng đảm bảo kỳ vọng với code ngắn hơn nhiều lần.

1.8 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Cơ sở dữ liệu	Chỉ mục B ⁺ -cây, quét khoảng, khoá chính	Cùng cấu trúc, và lý do chọn nó chính là mục 6. Điểm đáng nhớ: chỉ mục bám nhanh hơn cho truy vấn bằng nhưng vô dụng cho BETWEEN — đó là vì sao B-cây là mặc định.
Hệ tệp	Chỉ mục thư mục, ánh xạ khối	Cùng ràng buộc đĩa nên cùng lời giải. Ví dụ về việc một mô hình chi phí (chuyển khối) sinh ra một họ cấu trúc riêng — xem thêm D/24.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Hình học tính toán	Cây khoảng, cây phân đoạn, cây k -d	Đều là cây tìm kiếm được <i>tăng cường</i> : thêm thông tin ở mỗi nút để trả lời câu hỏi hình học. Cùng điều kiện: thông tin phải tổng hợp được từ các con ($D/22$).
SLAM và xử lý đám mây điểm	Cây k -d cho tìm điểm gần nhất trong ICP	Cùng ý tưởng chia không gian theo ngưỡng, chỉ khác là chia luân phiên theo từng chiều toạ độ. Cùng điểm yếu: dữ liệu phân bố lệch làm cây mất cân bằng, nên trong thực tế phải xây lại định kỳ.
Đồ hoạ, dò tia	Cây bao khối (BVH), cây phân vùng không gian	Cùng mục tiêu: giảm số phép kiểm tra từ tuyến tính xuống lôgarit bằng cấu trúc phân cấp. Tiêu chí xây cây khác nhau (diện tích bề mặt thay vì cân bằng số nút).
Lập trình hàm	Cấu trúc bền vững, cây bất biến	Cây rất hợp với biểu diễn bền vững vì một thao tác chỉ đụng $O(\log n)$ nút, phần còn lại chia sẻ được giữa các phiên bản[3]. Đây là nền của cây bền vững ở $B/10$.
Hệ thống nhiều luồng	Danh sách bỏ qua trong bộ nhớ đệm ghi của cơ sở dữ liệu	Lý do chọn không phải tốc độ mà là <i>để làm phiên bản đồng thời</i> : các thao tác chỉ động vào con trỏ cục bộ, không cần quay cả cây.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì trường hợp xấu nhất của nó không hiếm mà lại rất phổ biến: dữ liệu thật thường tới theo trật tự nào đó — theo thời gian, theo mã tăng dần, theo thứ tự đã sắp từ nguồn khác. Khi đó mọi phần tử rẽ cùng một hướng và cây thành danh sách với chiều cao n . Đây là cùng dạng vấn đề với bảng băm ở $B/08$: “trung bình nhanh” vô nghĩa nếu phân phối thực tế nằm đúng vào vùng xấu.
- A2. Vì hai bối cảnh có đơn vị chi phí khác nhau. Trong bộ nhớ, chi phí là số phép so sánh và số lần lần theo con trỏ, nên cây nhị phân là hợp lý; giữa AVL và đỏ-đen thì đỏ-đen quay ít hơn nên tốt hơn khi có nhiều chèn/xoá, và đó là lựa chọn của thư viện chuẩn. Trên đĩa, chi phí là số lần đọc khối và một lần đọc đắt hơn hàng triệu phép so sánh; lúc đó phải làm nút thật to để cây thật thấp — B-cây. Cùng bài toán, đổi mô hình chi phí, đổi cấu trúc tối ưu.
- A3. Vì hình dạng của treap chỉ phụ thuộc vào thứ tự tương đối của các độ ưu tiên, chứ không phụ thuộc thứ tự chèn. Mà độ ưu tiên do bạn sinh ngẫu nhiên, nên treap có phân phối hình dạng giống hệt một BST được chèn theo thứ tự ngẫu nhiên đều — và chiều cao kỳ vọng của cây đó là $O(\log n)$. Điểm mấu chốt: đổi thủ chọn được dữ liệu nhưng không chọn được độ ưu tiên, nên không dựng được trường hợp xấu.
- A4. Vì luật cân bằng thì bạn hầu như luôn dùng lại từ thư viện, còn tăng cường là thứ bạn phải tự nghĩ cho từng bài. Nó biến một cấu trúc chỉ biết “có hay không” thành cấu trúc trả lời được “phần tử thứ k ”, “bao nhiêu phần tử nhỏ hơn x ”, “đoạn nào chứa điểm này”. Điều kiện áp dụng cũng rất dễ kiểm: thông tin thêm phải tổng hợp được từ hai con trong $O(1)$ — và chính điều kiện đó sẽ trở lại ở $B/10$ dưới tên “phép toán có tính kết hợp”.
- A5. Khi dữ liệu tĩnh hoặc gần như tĩnh. Mảng đã sắp cho tìm kiếm nhị phân với truy cập gần như tuần tự, rất hợp cache, không có con trỏ và không có chi phí cấp phát; cây cân bằng lần theo con trỏ nên mỗi tầng có thể là một lần trượt cache. Với vài triệu phần tử, khác biệt hằng số này lớn hơn khác biệt giữa $O(\log n)$ và $O(n)$ chèn — miễn là số lần chèn nhỏ. Quy tắc thực dụng: ít cập nhật thì mảng đã sắp; cập nhật liên tục thì cây.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng một cuốn danh bạ. Nếu tên được ghi lộn xộn, muốn tìm ai bạn phải đọc từ đầu. Nếu ghi theo thứ tự chữ cái, bạn mở giữa quyển rồi lật qua lật lại — đó là tìm kiếm nhị phân, rất nhanh, nhưng thêm một cái tên mới vào giữa thì phải chép lại cả quyển.

Cây tìm kiếm là cách sắp xếp danh bạ thành nhiều tờ rồi có mũi tên chỉ nhau, để thêm một tên chỉ phải sửa vài tờ. Vấn đề duy nhất: nếu bạn thêm tên theo đúng thứ tự chữ cái từ A tới Z, các mũi tên sẽ nối thành một hàng dài và bạn quay lại đúng tình cảnh đọc từ đầu.

Có hai cách chống. Cách thứ nhất là cứ vài lần thêm lại chỉnh cho cây khỏi lệch — giống việc sắp lại giá sách cho hai bên đều nhau. Cách thứ hai thông minh hơn và rất đáng nhớ: gán cho mỗi tờ một con số ngẫu nhiên và xếp cây theo con số đó thay vì theo thứ tự bạn thêm vào. Vì con số là do bạn tung ra, người khác không thể cố tình đưa dữ liệu theo thứ tự xấu để hại bạn.

Cuối cùng là một chuyện thực tế thú vị. Nếu quyển danh bạ để trên bàn, việc lật một trang gần như miễn phí. Nhưng nếu nó nằm trong kho và mỗi lần cần một trang thì phải chạy xuống kho lấy, thì bạn sẽ không muốn mỗi trang chỉ có một cái tên — bạn sẽ muốn mỗi lần chạy xuống kho mang lên một tập dày. Đó chính là lý do cơ sở dữ liệu không dùng cây nhị phân mà dùng cây có nút rất to.

Một câu: *Chiều cao là thứ phải giữ; cách trả tiền cho nó — quay tay hay tung ngẫu nhiên — là chỗ các cấu trúc khác nhau.*

Chỗ không rút gọn được: Việc chọn thông tin tăng cường cho một bài cụ thể vẫn là bước sáng tạo; điều kiện “tổng hợp được từ hai con” chỉ nói cái gì không dùng được.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vấn đề gốc của BST	Dữ liệu vào theo thứ tự đã sắp làm cây thoái hoá thành danh sách, chiều cao $O(n)$. Đây là trường hợp phổ biến, không hiếm.
••• Phép quay làm gì	Bảo toàn thứ tự duyệt giữa (bất biến ngữ nghĩa) trong khi thay đổi chiều cao (bất biến cấu trúc). Là thao tác cơ bản duy nhất của mọi cây tự cân bằng.
••• Vì sao treap cân bằng	Hình dạng chỉ phụ thuộc thứ tự các độ ưu tiên ngẫu nhiên, không phụ thuộc thứ tự chèn $\Rightarrow$ tương đương BST chèn ngẫu nhiên $\Rightarrow$ chiều cao kỳ vọng $O(\log n)$.
••• Tăng cường — điều kiện	Thông tin thêm phải tổng hợp được từ hai con trong $O(1)$: kích thước, tổng, cực trị được; “phần tử phổ biến nhất” thì không.
••• Cây thống kê thứ tự	Lưu kích thước cây con $\Rightarrow$ trả lời “phần tử thứ k ” và “bao nhiêu phần tử nhỏ hơn x ” trong $O(\log n)$.
••• Vì sao B-cây tồn tại	Chi phí thật trên đĩa là số lần đọc khối, không phải số phép so sánh. Nút to $\Rightarrow$ chiều cao $\log_B n \Rightarrow 3-4$ lần đọc cho một tỉ bản ghi.
•• AVL so với đỏ-đen	AVL chặt hơn nên cây thấp hơn, tìm nhanh hơn, nhưng quay nhiều hơn khi cập nhật. Đỏ-đen lỏng hơn, ít quay hơn — lựa chọn của thư viện chuẩn.
•• Splay tree	Không đảm bảo từng thao tác, chỉ $O(\log n)$ khấu hao; phần tử hay truy cập tự nổi lên gần gốc. Nhược điểm: thao tác đọc cũng làm đổi cấu trúc.
•• Danh sách bỏ qua	Nhiều tầng danh sách liên kết, xác suất lên tầng $1/2$; cùng đảm bảo kỳ vọng với cây, dễ cài và dễ làm phiên bản nhiều luồng.
•• Mẫu ngẫu nhiên hoá lặp lại	Băm phổ dụng (B/08), treap (chương này), chốt quicksort (C/14) — cùng một nguyên lý: chuyển sự phụ thuộc từ dữ liệu sang nguồn ngẫu nhiên bạn kiểm soát.
•• Cây khoảng	Lưu cực đại của điểm kết thúc trong cây con $\Rightarrow$ hỏi “đoạn nào chứa x ”, “đoạn nào giao $[l, r]$ ”.
• Khi nào mảng đã sắp thẳng cây	Dữ liệu tĩnh hoặc ít cập nhật: không con trỏ, hợp cache, không cấp phát. Cây chỉ thẳng khi có cập nhật liên tục.
• Lời khuyên thi đấu	Đừng cài AVL hay đỏ-đen; cần cây tự viết thì dùng treap (đặc biệt khi cần tách/ghép), còn lại dùng thư viện chuẩn.
•• Splay và phỏng đoán tối ưu động	Splay không lưu thông tin cân bằng nào, vẫn đạt tối ưu tĩnh và tính chất tập làm việc. Phỏng đoán “cạnh tranh hằng số với mọi cây nhị phân” vẫn mở; tiến bộ lớn nhất là Tango tree, $O(\log \log n)$ -cạnh tranh.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào tôi thực sự cần tự cài cây cân bằng thay vì dùng thư viện?

| Chủ yếu khi cần hai thao tác mà thư viện chuẩn không có: *tách* cây thành hai theo khoá và *ghép* hai cây, hoặc cần truy vấn “phần tử thứ k ”. Với nhu cầu thứ nhất, treap là lựa chọn gần như bắt buộc vì tách/ghép trên treap chỉ vài dòng. Với nhu cầu thứ hai, nếu khoá là số nguyên trong khoảng vừa phải thì Fenwick ($B/10$) đơn giản hơn nhiều và nhanh hơn — hãy kiểm tra khả năng đó trước khi viết cây.

2. Tôi nghe nói cây đỏ-đen “khó cài”. Khó ở chỗ nào?

| Ở phần xoá. Chèn có bốn trường hợp và còn dễ nhớ; xoá có nhiều trường hợp hơn, phải xử lý cả tình huống “nút thiếu màu đen” lan ngược lên gốc, và mỗi trường hợp lại chia theo màu của anh em và của các cháu. Sai một nhánh thì cây vẫn chạy đúng về mặt thứ tự nhưng mất cân bằng dần — một lỗi im lặng rất khó phát hiện. Đây là lý do thực tế khiến treap và danh sách bỏ qua được ưa chuộng ngoài các thư viện chuẩn.

3. Cây k -d có phải là cây tìm kiếm không, và vì sao nó hay mất cân bằng?

| Nó là cây tìm kiếm nhị phân trên không gian nhiều chiều: mỗi tầng chia theo một chiều toạ độ khác nhau. Nó mất cân bằng vì chiều được chọn luân phiên chứ không theo phân bố dữ liệu; nếu điểm dồn về một mặt phẳng hoặc một trục thì nhiều lần chia gần như không tách được gì. Trong thực tế — ví dụ tìm điểm gần nhất trên đám mây điểm của cảm biến — cách xử lý phổ biến là chọn chiều có phương sai lớn nhất thay vì luân phiên, và xây lại cây định kỳ thay vì cập nhật từng điểm.

4. Cây cân bằng có dùng được trong môi trường nhiều luồng không?

| Được nhưng khó, vì phép quay động tới nhiều nút cùng lúc nên phải khoá một vùng lớn. Có ba hướng thực tế: dùng danh sách bỏ qua, vốn chỉ cần sửa vài con trỏ cục bộ và có bản không khoá; dùng cấu trúc bền vững kiểu lập trình hàm, ở đó người đọc luôn nhìn một ảnh chụp bất biến[3]; hoặc dùng B-cây với kỹ thuật khoá theo nút và tách trước khi đi xuống. Splay tree là lựa chọn tệ nhất ở đây vì ngay cả thao tác đọc cũng làm thay đổi cấu trúc.

5. Vì sao đếm nghịch thế lại là bài tiêu biểu cho chương này?

| Vì nó có ba lời giải bằng ba công cụ khác nhau của phần B, và so sánh chúng dạy nhiều hơn bản thân bài. Cách một: cây thống kê thứ tự, chèn dần từ trái sang phải và mỗi lần hỏi “bao nhiêu phần tử đã chèn lớn hơn giá trị này”. Cách hai: Fenwick trên giá trị đã nén ($B/10$) — ngắn hơn và nhanh hơn. Cách ba: sắp xếp trộn, đếm trong lúc trộn ($C/14$). Ba cách cùng $O(n \log n)$ nhưng hằng số và độ dài code rất khác.

6. “Bền vững” nghĩa là gì và vì sao cây lại hợp với nó?

| Bền vững là khả năng giữ lại mọi phiên bản cũ sau khi cập nhật, để truy vấn được quá khứ. Cây rất hợp vì một thao tác chỉ đụng tới các nút trên một đường từ gốc xuống lá — chỉ $O(\log n)$ nút. Thay vì sửa tại chỗ, ta tạo bản sao của đúng đường đó và cho các nhánh còn lại trở chung sang cây cũ; chi phí thêm là $O(\log n)$ bộ nhớ mỗi phiên bản. Ý tưởng này sẽ quay lại với cây phân đoạn bền vững ở $B/10$.

7. Nếu khoá là số nguyên trong khoảng hẹp, có gì tốt hơn cây không?

| Có, và chênh lệch lớn. Với khoá trong $[0, U)$, cấu trúc van Emde Boas cho mọi thao tác trong $O(\log \log U)$ — nhanh hơn hẳn $O(\log n)$ khi U không quá lớn, đổi lại tốn bộ nhớ $O(U)$ và cài đặt phức tạp. Trong thực hành thi đấu, thường chỉ cần một Fenwick trên mảng bit hoặc một cấu trúc phân tầng đơn giản là đủ nhanh. Bài học chung vẫn thế: biết thêm về dữ liệu — ở đây là “khoá là số nguyên nhỏ” — thì luôn mua được tốc độ.

I Bài tập (mức: ● khởi động ● ● chuẩn ● ● ● khó)

Bài	Nguồn	Điều cần nhận ra
• Cài BST cơ bản, rồi chèn $1..n$ theo thứ tự và đo chiều cao	Bài tập cài đặt	Thấy tận mắt sự thoái hoá; sau đó chèn theo thứ tự ngẫu nhiên và đo lại.
• Dùng tập hợp có thứ tự để tìm phần tử gần nhất với mỗi truy vấn	CSES “Concert Tickets”[4]	Bài mẫu cho việc dùng <code>lower_bound</code> — thứ mà bảng băm không làm được.
•• Đếm số nghịch thế	CSES “Inversions”-loại; bài kinh điển	Giải bằng cả ba cách (cây thống kê thứ tự, Fenwick, sắp xếp trộn) và so hằng số.
•• Cài treap với chèn, xoá, tìm phần tử thứ k	Bài tập cài đặt	Bắt đầu bằng tách/ghép; mọi thao tác khác viết từ hai thao tác đó.
•• Duy trì trung vị của một tập thay đổi bằng cây thống kê thứ tự	Bài tập kinh điển	So sánh với cách hai heap ở B/07 — cây tổng quát hơn, heap nhanh hơn cho đúng bài này.
•• Lịch đặt phòng: kiểm tra chồng lấn khi thêm mỗi lịch	LeetCode “My Calendar I/II”	Cây khoảng hoặc tập hợp có thứ tự; chú ý xử lý biên đóng/mở cho đúng.
••• Cài danh sách bỏ qua và so tốc độ với cây cân bằng của thư viện	Bài tập thực nghiệm	Đo cả thời gian lẫn bộ nhớ; giải thích chênh lệch bằng cache chứ không bằng độ phức tạp.
••• Dùng treap để chèn/xoá cả đoạn của một dãy	Bài tập thi đấu kinh điển	Treap theo chỉ số ngẫu nhiên — ứng dụng mạnh nhất của treap và là lý do chính để tự cài nó.
••• Cài B-cây bậc lớn và đo số lần truy cập nút so với cây nhị phân	Bài tập hệ thống	Mô phỏng chi phí đọc khối; đây là cách rẻ nhất để thấy vì sao cơ sở dữ liệu chọn B-cây.
••• Cây k -d cho tìm điểm gần nhất trên đám mây điểm 3D	Bài tập ứng dụng	So với vét cạn về thời gian; thử phân bố lệch để thấy cây mất cân bằng.

Kết chương và đọc tiếp

Cây tìm kiếm là câu chuyện về một bất biến: chiều cao phải là $O(\log n)$. Mọi cấu trúc trong chương chỉ khác nhau ở cách trả tiền cho bất biến đó — quay theo luật chặt, quay theo luật lỏng, quay theo độ ưu tiên ngẫu nhiên, hay làm nút to ra cho hợp với đơn vị chi phí của đĩa. Kỹ thuật đáng mang theo nhất không phải luật cân bằng mà là *tăng cường*, cùng điều kiện của nó: thông tin thêm phải tổng hợp được từ các con.

Chương tiếp theo lấy đúng ý tưởng tăng cường ấy và đẩy tới cùng cho một bài toán rất cụ thể: truy vấn trên đoạn kèm cập nhật. Bạn sẽ gặp lại điều kiện “tổng hợp được từ các con” dưới cái tên chính xác của nó — tính kết hợp — và sẽ thấy hai cấu trúc có tỉ lệ sức mạnh trên độ dài code cao nhất trong cả phần B là Fenwick và cây phân đoạn (B/10).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Sleator & Tarjan, “Self-Adjusting Binary Search Trees”	Journal of the ACM, 1985	Bài gốc của splay tree. Đọc phần phát biểu tối ưu tĩnh và tập làm việc; phần chứng minh là ví dụ mẫu mực cho phương pháp thế năng (A/03).
nền tảng	Demaine, Harmon, Iacono, Pătraşcu, “Dynamic Optimality—Almost”	SIAM Journal on Computing, 2007	Tango tree và cận dưới đàn xen. Đọc mục giới thiệu để hiểu phát biểu của phỏng đoán tối ưu động, kể cả khi bỏ qua chi tiết kỹ thuật.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Knuth, “Optimum Binary Search Trees”	Acta Informatica, 1971	Cây tìm tối ưu tính được bằng quy hoạch động $O(n^2)$ nhờ tính đơn điệu của điểm chia — cùng kết quả sẽ trở lại ở C/16.
tra khi cần	Demaine, khoá học MIT 6.851 <i>Advanced Data Structures</i>	tài liệu khoá học công khai, MIT	Bài giảng về cây bền vững, tối ưu động và các cấu trúc gần biên; hợp để đi tiếp sau chương này.

Nguồn

- [1] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [2] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [3] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [4] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).